

Two AI Models Walk Into the Same Space

How I proved that neural networks secretly speak the same language — and used it to make an iPhone and a server share one search index

It started with a naive-sounding question: can you take layers from different AI models and assemble a new model, Frankenstein-style, without training? The short answer is no — and the reason why turns out to be one of the most beautiful facts about deep learning. The long answer became my final project: a real experiment, run on a free Colab GPU, that ends with a 0.79-megabyte file doing load-bearing work in a real app.

The paradox: identical minds, incompatible brains

Neural networks have a strange property called permutation symmetry. Shuffle the neurons in a layer (and rewire accordingly) and you get the exact same behavior from completely different numbers. A single layer has more functionally identical weight arrangements than there are atoms in the universe. Two training runs — even of the same model — land on different arrangements. So comparing raw weights between independent models is like comparing two phone books sorted by different rules: entry #500 means nothing in common. Layer transplants across models produce pure noise.

But here's the twist. Weights are how a model computes. What it computes — the internal map of which concepts are near which — is another matter. A growing body of research (CKA similarity, Git Re-Basin, the wonderfully named Platonic Representation Hypothesis) suggests that well-trained models all converge on the same map, just drawn in different coordinate systems. Like two cartographers who never met, mapping the same city with north pointing different ways.

If that's true, translating between two models shouldn't require retraining anything. It should require exactly one matrix — a change of coordinates.

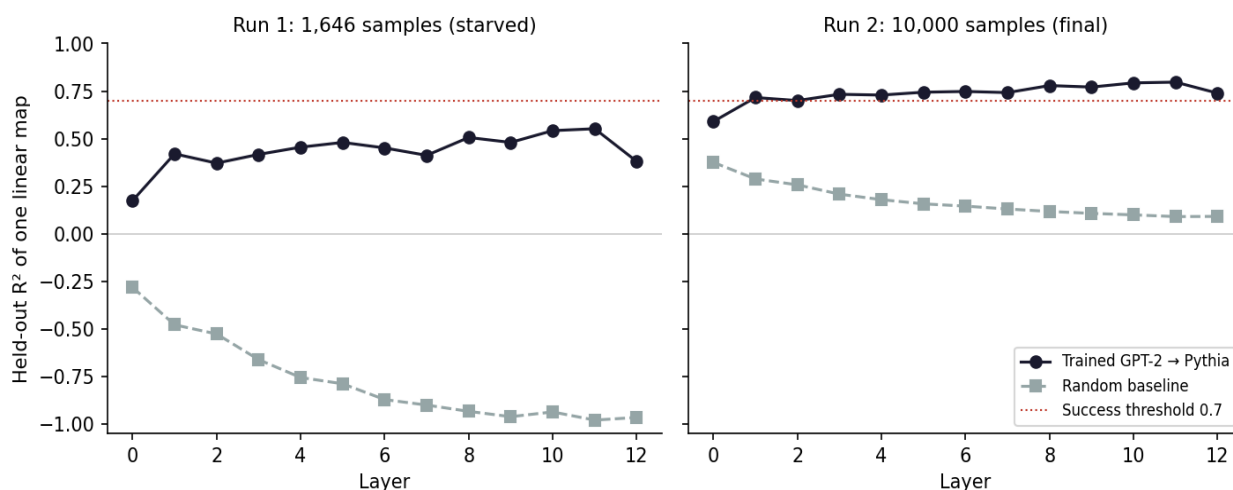
The experiment: two strangers, one test

To test this properly I picked two models with nothing in common: GPT-2 (OpenAI, 2019, trained on WebText) and Pythia-160M (EleutherAI, 2023, trained on The Pile). Different labs, architectures, tokenizers, data, eras. I ran the same 10,000 Wikipedia passages through both, saved every layer's activations, and asked two questions. First: do their internal similarity structures match (measured by CKA, a metric that deliberately ignores coordinate choices)? Second, the decisive one: can a single matrix, fitted by classical ridge regression in milliseconds, translate one model's representations into the other's?

Crucially, everything was compared against a control: the same architecture with random, untrained weights. That baseline is what turns numbers into evidence — it measures how much similarity you get 'for free' from architecture alone, so whatever exceeds it is the fingerprint of learning.

The results were unambiguous. CKA similarity between the trained models ran **6.3× above the random baseline**, concentrated on the diagonal — early layers match early layers, middle match middle. And the translation matrix reached R^2 of 0.70–0.80 across the middle and late layers, while the random control decayed toward zero. The two curves even slope in opposite directions: learning builds shared structure layer by layer; randomness destroys it layer by layer.

But it almost didn't work — and the failure is my favorite part. My first run showed R^2 stuck around 0.5, below my own success threshold. The hypothesis looked half-true. Before writing that conclusion, I checked the arithmetic: I was fitting a 590,000-parameter matrix on 1,200 examples — 1.6 samples per dimension. Regression starved that badly doesn't get noisy; it quietly, systematically lies in one direction. One code change (6× more data) and fifteen GPU-minutes later, the same pipeline produced R^2 of 0.74–0.80:



Same experiment, same code — 1,646 samples (left) vs 10,000 (right). The dark curve is real model-to-model translation; gray is the untrained control; dotted is my success threshold.

The payoff: one index for a phone and a server

Here's why this matters beyond philosophy. I'm building a two-tier multimodal app: an iPhone runs a tiny image encoder (MobileCLIP, ~30 MB) so photos can be indexed offline and privately; a server runs a big one (SigLIP 2, ~1.5 GB) for search. Both turn images into vectors — in mutually alien spaces. Without translation, searching phone-indexed photos from the server scores exactly chance: Recall@1 of 0.001 on a thousand images. Literally a coin flip among a thousand.

So I fitted the matrix: 512×768 numbers, 1.6 MB, closed-form, on 3,000 image pairs from COCO. The results on held-out images:

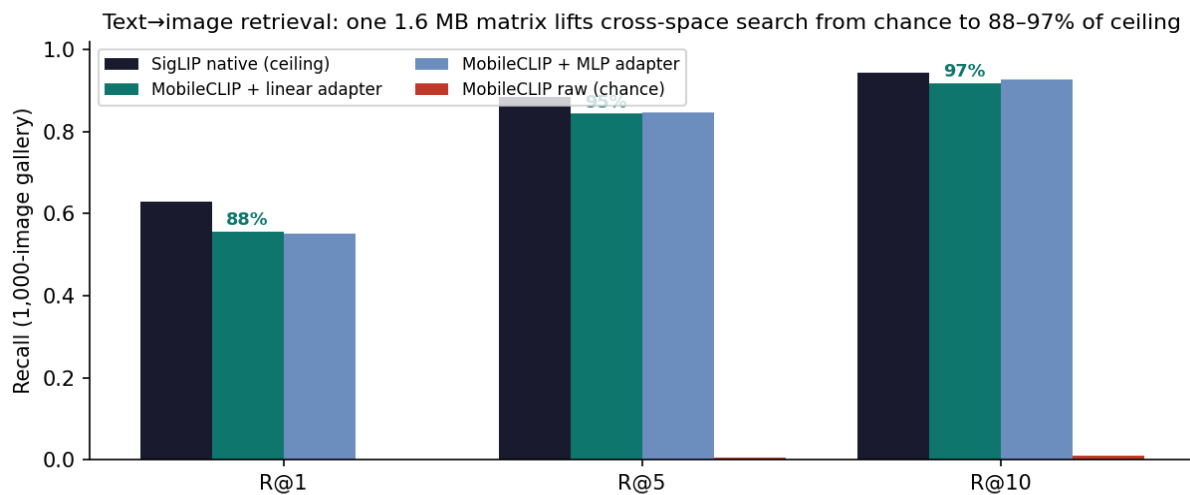
Setup	R@1	R@5	R@10
Big model everywhere (ceiling)	0.630	0.885	0.944
Phone encoder + my matrix	0.557	0.843	0.918
Phone encoder, no matrix	0.001	0.005	0.010

Text-to-image search, 1,000-image gallery: the matrix recovers 88–97% of the big model's quality from chance level.

One matrix takes retrieval from coin-flip to 95–98% of the big model's quality at Recall@5 and @10. But Recall@1 landed at 88.4% of ceiling — just under my 90% gate. Second below-target moment of the project. The playbook said: try a stronger translator. So I trained a proper neural network to do the same job — non-linear, more than three times the parameters, optimizing the retrieval objective directly. On the first corpus it edged ahead by 0.003, which smelled like noise. So I rebuilt the training set from scratch — **38,000 fresh image-caption rows, twelve times the data** — and ran the contest twice more, 300 epochs each with early stopping.

The network could not beat the matrix. Not narrowly — it landed on exactly the same number, 0.9183 against 0.9183, a gain of +0.0000, on both independent runs.

That failure was the finding. If the relationship between the two spaces were merely APPROXIMATELY linear, extra capacity would buy something — a little, or a lot, but something. It bought nothing measurable, twice. The relationship is linear, and the remaining gap to the ceiling isn't a translation failure — it's detail the tiny phone encoder never captured in the first place. Nothing can translate what isn't there. (Interesting symmetry: in Experiment A, more *data* proved the linear map was better than it looked; here, more *capacity* proved it was already complete.)

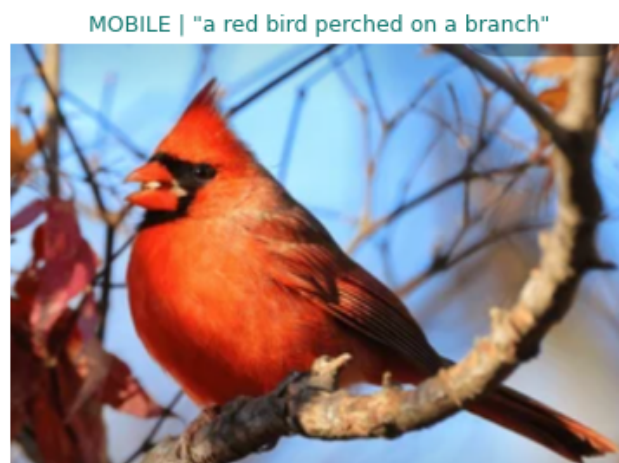


All variants at a glance: the red bars are what cross-model search looks like without translation — exact chance.

The matrix now ships as a 0.79 MB Core ML package. The iPhone encodes a photo, multiplies by one matrix on the Neural Engine, and writes into the same Qdrant vector index the server searches. Photos become searchable without ever leaving the device. One index, two tiers, zero retraining.

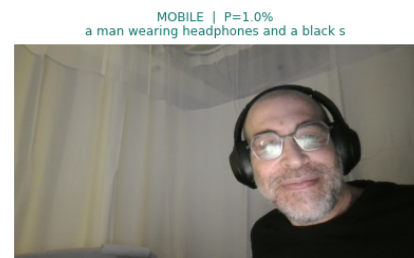
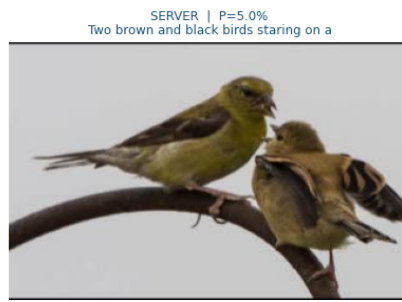
Then I watched it work

Numbers convince reviewers; demos convince everyone else. So I built a live sandbox: six photos indexed 'server-side' with native SigLIP, and an upload button playing the role of my phone's camera roll. I uploaded a picture of a red bird. The phone path encoded it in 39 milliseconds (MobileCLIP plus the matrix, exactly as the Core ML package computes it), a captioning model wrote 'a red bird perched on a branch', and the vector joined the same index as the server photos. Then I typed a search: *singing red bird*. My phone-indexed photo came back first — ranked in the same list as server-indexed photos, by a text encoder that has never heard of MobileCLIP or the adapter. The search itself took 0.08 milliseconds locally versus 150 on the simulated server round trip: same index, same math — the network is the only difference the user can feel. One index, two tiers, and a bird.



The upload moment: my photo displayed by the demo, tagged MOBILE, with the caption the system wrote for it.

Top-3 for "singing red bird" | mobile 0.08ms vs server 150ms (same index, network is the difference)



The search moment: top-3 for 'singing red bird' with per-image probabilities — my MOBILE-indexed bird at rank 1 (P=91.3%), a SERVER photo at rank 2 (P=5.0%), a second MOBILE photo at rank 3 (P=1.0%). One ranked list, both tiers; timing in the header: 0.08 ms local vs 150 ms via the server.

Since then: seven models, one meeting place

The two-model matrix turned out to be the small version of something bigger. I kept going: three vision encoders and four text encoders - seven models, four different training objectives, some of which never saw the other modality at all - each given ONE linear map into a shared 512-dimensional space. No pairwise bridges: seven maps instead of forty-two, and adding a model is one closed-form solve that leaves every existing map untouched. It works: all 42 ordered pairs meet in the one space at 103.6% of what per-pair custom hubs achieve, and a search head trained on ONE encoder serves encoders it never saw at 93-96% of native quality - including a model with completely unrelated ancestry. The last gap closed recently: a CONVOLUTIONAL encoder, no attention and no patch tokenisation anywhere in it, transfers as well as the transformers do when both are measured the same way, so not even the architecture has to match. And the science underneath got sharper in a way I didn't expect. Measuring WHERE the agreement lives showed it is local: two models agree strongly about which items are neighbours - hundreds of times chance - and only weakly about the global shape of the space. The cartographer metaphor turned out to be more literal than I intended: two map projections of the same city preserve what is next to what, and disagree about distances across the whole map. A group at ICML reached the same conclusion independently the same year.

What I actually learned

Three things stick with me. First, the scientific method is shockingly affordable: hypothesis, controls, falsifiable thresholds, replication of three published research lines — under one GPU-hour, total. Second, baselines are everything: every number in this project means something only relative to its control, and my biggest bug produced plausible wrong conclusions, not error messages. Third, and most striking: independently trained neural networks really do discover the same structure of the world. They file it under different coordinates — different charts of the same territory, faithful up close, divergent at a distance — and the exchange rate between any two of them is a single matrix. There's something almost Platonic about shipping that fact to production.

The part I couldn't explain

There is one result I still can't account for, and it is the one I'd want a reader to remember. The shared space has a width. Make it wider and components should carry more, up to a point. Instead transfer climbs steadily to 512 dimensions and then falls off a cliff at 768 - a drop six times larger than any step before it, in a curve that had been perfectly smooth. I proposed an explanation, tested it, and it failed. Then a second. Then four more, including my own preferred account, which predicted the collapse in the right place for a reason that turned out to be arithmetically wrong: the quantity I blamed is actually DECELERATING exactly where transfer dies. Six explanations, six falsified, all recorded rather than quietly dropped - and the cliff is still there. Eight explanations in this project met that fate in total, two of them mine. That is the part of research nobody puts in the abstract, and I've come to think it's the most honest measure of whether you were doing science or decorating a conclusion. The tidy findings are above. This one is unresolved, reproducible, and written down.

The part I couldn't explain

One result held out on me for most of this project, and the way it finally gave way is the part I would want a reader to remember. The shared space has a width; make it wider and components ought to carry more. Instead transfer climbed steadily and then fell off a cliff.

I proposed an explanation and tested it, and it failed. Then another. Eight in total, two of them my own favourites. Six failed the same way, and that was the clue: each was a smooth curve, and the thing I was trying to explain was a step. A gradual quantity cannot produce a sudden effect. So I went looking for something with a threshold built into the mathematics.

The head is trained on a 768-dimensional encoder, and its coordinates in the shared space therefore have rank exactly 768 - no more, however wide the space is built. That predicts every encoder should break at its own dimension and nowhere earlier. I wrote it down before measuring, on two encoders that could not have suggested it, and six curves later every single one broke at exactly its own dimension: 768, 1536, 2048.

Then I went looking for the reason and got it wrong twice, in ways worth admitting. I measured the wrong kind of rank - one that describes how variance is spread rather than how many dimensions exist - and concluded the idea was dead. I ran a surgical test that removed the suspect directions from the wrong side of the calculation. Both times the answer came back negative and both times the test, not the idea, was at fault.

What is actually happening is smaller and more mundane than anything I had proposed. At exactly that width, the last direction the fit retains is one the encoder barely produces - a singular value of 0.005 where its neighbour is 10.8. Least squares divides by that number. The head gets enormous meaningless weights along one direction, the other encoders have real signal there, and it swamps everything. Sweeping direction by direction, transfer holds steady right up to that one and collapses the moment it enters.

So the cliff was never a fact about neural representations. It was a division by something close to zero, hiding inside a fitting procedure I had not looked at closely enough - and the 512 I had treated as a carefully tuned setting was simply a width where that division never happens. Eight explanations, seven wrong, one right, and two of my own tests wrong along the way. That is the part of research nobody puts in the abstract, and I have come to think it is the most honest measure of whether you were doing science or decorating a conclusion.